

Sponser Analítica

Pipeline de datos, pronóstico de demanda y reabastecimiento productivizados para un distribuidor de nutrición deportiva

Trabajo Fin de Máster	Máster Data Science, Big Data & Business Analytics 2025-2026 (Semi-presencial)
Universidad	Universidad Complutense de Madrid (UCM), Instituto NTIC
Autor	Joseph Alejandro Mora Murillo
Tutores	Carlos Ortega, Santiago Mota
Repositorio	https://github.com/0107joseph-ui/sponser-analitica-tfm (privado, tutores con acceso)

Índice

1. Resumen ejecutivo
2. Contexto y objetivos de negocio
3. Fuente de datos y gobernanza
4. Arquitectura de la solución
5. Pipeline de datos (ETL)
6. Análisis exploratorio de datos
7. Modelado predictivo de demanda
8. Reabastecimiento: de la predicción a la decisión de compra
9. Productivización: la aplicación web
10. Discusión: interpretabilidad y limitaciones
11. Conclusiones y trabajo futuro
12. Bibliografía y referencias
13. Anexo: código fuente del pipeline (ETL, modelado, reabastecimiento)

La documentación técnica ampliada de cada etapa está en `docs/arquitectura.md`, `docs/pipeline_datos.md` y `docs/modelado.md` del repositorio.

1. Resumen ejecutivo

Sponser Costa Rica es un distribuidor de nutrición deportiva con un catálogo de cerca de 170 SKUs y una cartera de 194 clientes activos. La operación dependía de reportes planos exportados a mano, sin un pronóstico de demanda que anticipara estacionalidad o eventos, y sin visibilidad sistemática del riesgo de que el inventario existente venza antes de poder venderse.

Este trabajo desarrolla, sobre datos reales de la empresa (ventas 2023-2026, inventario por lote con fecha de vencimiento, patrocinios), una solución de extremo a extremo: un pipeline de datos que limpia y agrega la información comercial, un modelo de pronóstico de demanda diaria por producto (LightGBM, arquitectura de dos etapas), una simulación de reabastecimiento consciente de vencimientos (FIFO), y una aplicación web donde el equipo comercial carga los datos actualizados, corre el modelo y analiza los resultados. Esto cumple el criterio de productivización de un modelo predictivo: recibe datos nuevos y devuelve una recomendación de compra lista para usar.

Cifras clave: 10.929 líneas de venta reales (abr. 2023 – may. 2026) · 194 clientes · ~170 SKUs · modelo con AUC de clasificación 0.90 y 93.4% de precisión agregada de volumen contra el backtest · 23% de los clientes concentran el 80% de la facturación.

2. Contexto y objetivos de negocio

La operación comercial de Sponser identificó tres problemas concretos que motivan este trabajo:

- **Falta de visibilidad consolidada:** ventas, clientes y patrocinios se revisaban en reportes planos exportados del sistema de facturación, sin agregación ni tendencia visible.
- **Reposición de inventario a criterio,** sin un pronóstico de demanda que distinguiera estacionalidad, eventos deportivos o inactividad de producto del ruido normal de la venta diaria.
- **Riesgo de vencimiento:** con inventario organizado por lotes con fecha de caducidad, no había forma sistemática de saber qué unidades, al ritmo de venta real, no se iban a consumir antes de vencer.

El objetivo del proyecto es doble: (a) demostrar un desarrollo analítico completo (descriptivo, predictivo y de decisión) sobre un problema de negocio real, y (b) entregar una herramienta que el equipo comercial de Sponser use de forma continua, no un análisis de una sola vez.

3. Fuente de datos y gobernanza

Los datos utilizados son reales y propiedad de Sponser Costa Rica, incluidos en este trabajo con autorización de la empresa para fines exclusivamente académicos (ver `LICENSE.md` del repositorio). No es un dataset público ni sintético: son los reportes de venta e inventario que la empresa exporta de su propio sistema de facturación.

Fuente	Contenido	Volumen
COM / Control Sponser (.xls)	Detalle de facturación por línea	10.929 filas · 3.071 facturas
Inventario (.xlsx)	Lotes con fecha de vencimiento	112 lotes activos
Rango temporal	—	29 abr. 2023 – 28 may. 2026
Clientes	—	194
Productos (SKUs)	—	~170

Diccionario de datos completo (columnas de cada tabla procesada) en el README del repositorio, sección "Diccionario de datos".

4. Arquitectura de la solución

La solución se divide en dos componentes independientes que se comunican solo a través de archivos y ejecución de subprocesos, nunca por importación de código. Así, el pipeline de datos y el backend pueden evolucionar sin arriesgarse mutuamente:

- **Pipeline de datos y modelado** (Python + pandas + LightGBM): extracción, transformación, entrenamiento del modelo, generación del pronóstico y cálculo de la recomendación de compra.
- **Backend + dashboard** (FastAPI + SQLAlchemy + frontend web): sirve los resultados del pipeline vía API, gestiona usuarios/roles, y permite disparar el pipeline desde la interfaz cuando llegan datos nuevos.

Diagrama de arquitectura y justificación detallada de cada decisión de diseño (capas de caché, seguridad, aislamiento entre componentes) en `docs/arquitectura.md`.

5. Pipeline de datos (ETL)

El pipeline resuelve tres retos particulares de los datos crudos de Sponser, antes de que cualquier análisis o modelo pueda usarlos:

5.1 Extracción

Los reportes de venta no son archivos Excel binarios: son una tabla HTML (GridView de ASP.NET) exportada con extensión `.xls`. Se parsean con la librería estándar de Python (`html.parser`), identificando cada archivo por su nombre para etiquetar la fuente (COM/Control) y el año.

5.2 Transformación

Se normaliza la moneda (un 0.04% de las filas viene en colones y se convierte a USD), se separan las salidas de inventario por patrocinio deportivo del resto de la venta comercial (no son venta orgánica y sesgarían el modelo si se incluyeran), y se calcula la venta neta restando el IVA del monto bruto facturado. Esto último se verificó empíricamente contra ambas fuentes de reporte, con 99.9% de coincidencia con la identidad contable esperada.

5.3 Carga

Las tablas resultantes (ventas diarias por producto, RFM de clientes, inventario por lote, patrocinios) se escriben como CSV, consumibles tanto por el modelo como por el backend sin conocer la lógica previa de extracción.

Detalle completo de cada transformación, con la lógica exacta de separación de patrocinios y conversión de moneda, en `docs/pipeline_datos.md`.

6. Análisis exploratorio de datos

La facturación mensual muestra una tendencia de crecimiento sostenida sobre los tres años de historia disponibles, con una estacionalidad marcada (picos y valles que se repiten en el mismo período del año), consistente con una demanda ligada a temporadas deportivas y patrocinios de eventos:

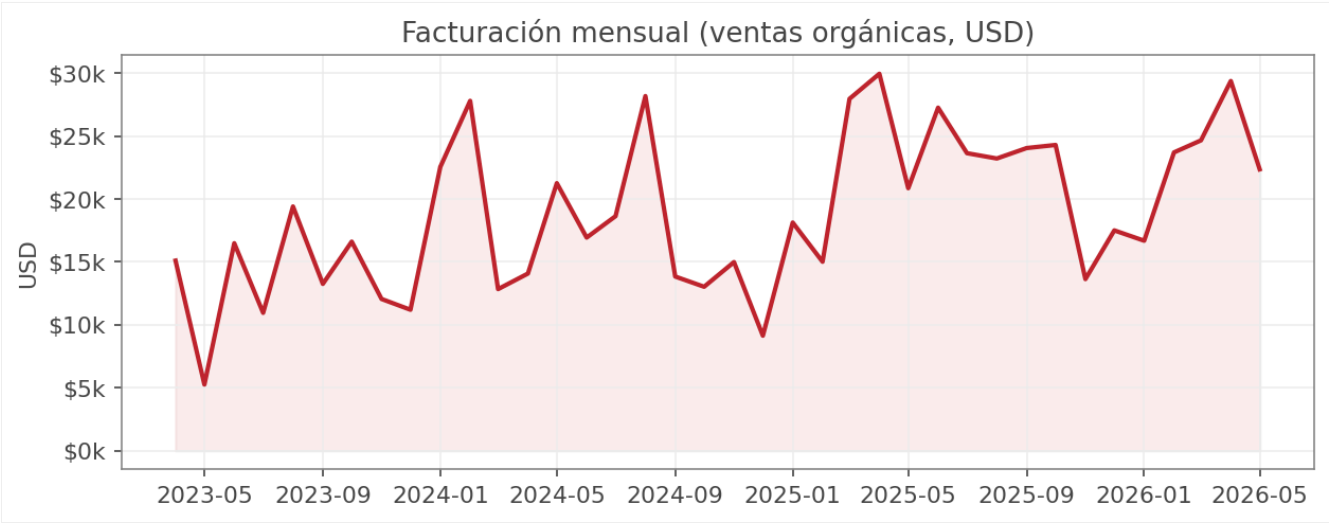


Figura 1. Facturación mensual de ventas orgánicas (excluye patrocinios), abril 2023 a mayo 2026.

La facturación está fuertemente concentrada en un núcleo de productos: 15 SKUs explican una porción desproporcionada de la venta total, lo que justifica priorizar el esfuerzo de pronóstico y reabastecimiento en ese subconjunto sin descuidar el resto del catálogo:

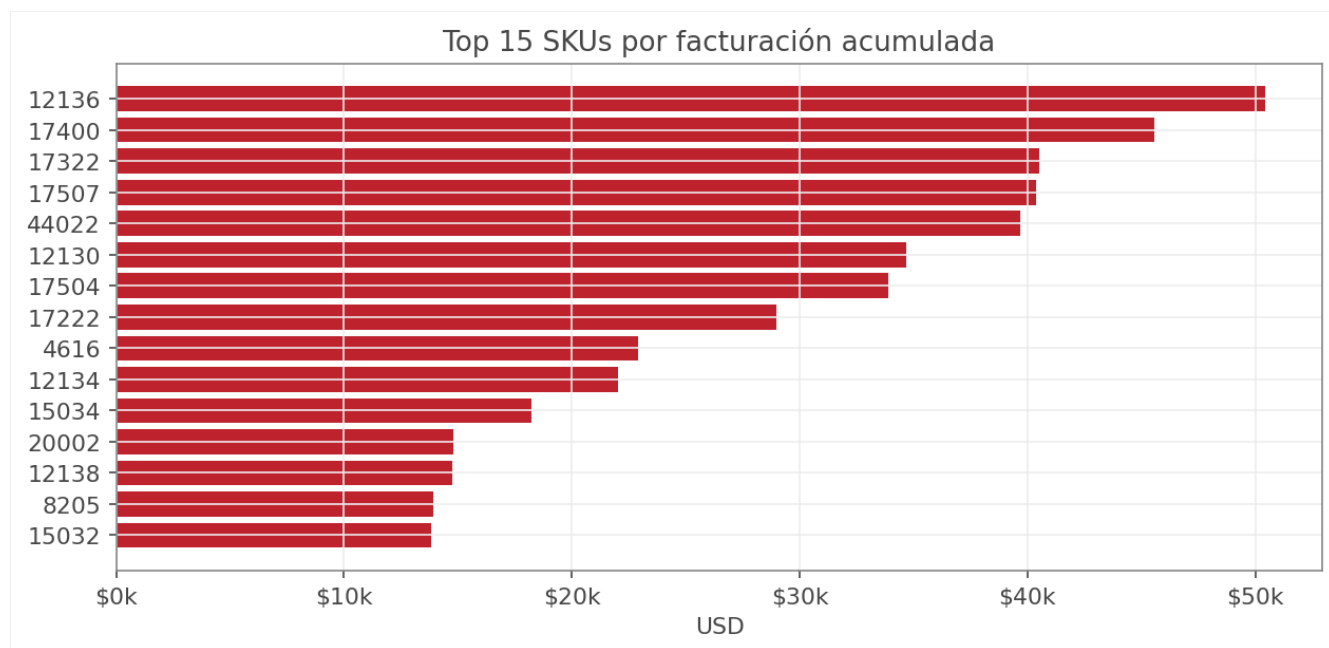


Figura 2. Los 15 SKUs de mayor facturación acumulada.

Lo mismo ocurre del lado de clientes: el **23% de la cartera concentra el 80% de la facturación**, un patrón de Pareto clásico que tiene implicaciones directas de negocio (priorización de cuentas clave, gestión de riesgo de concentración):

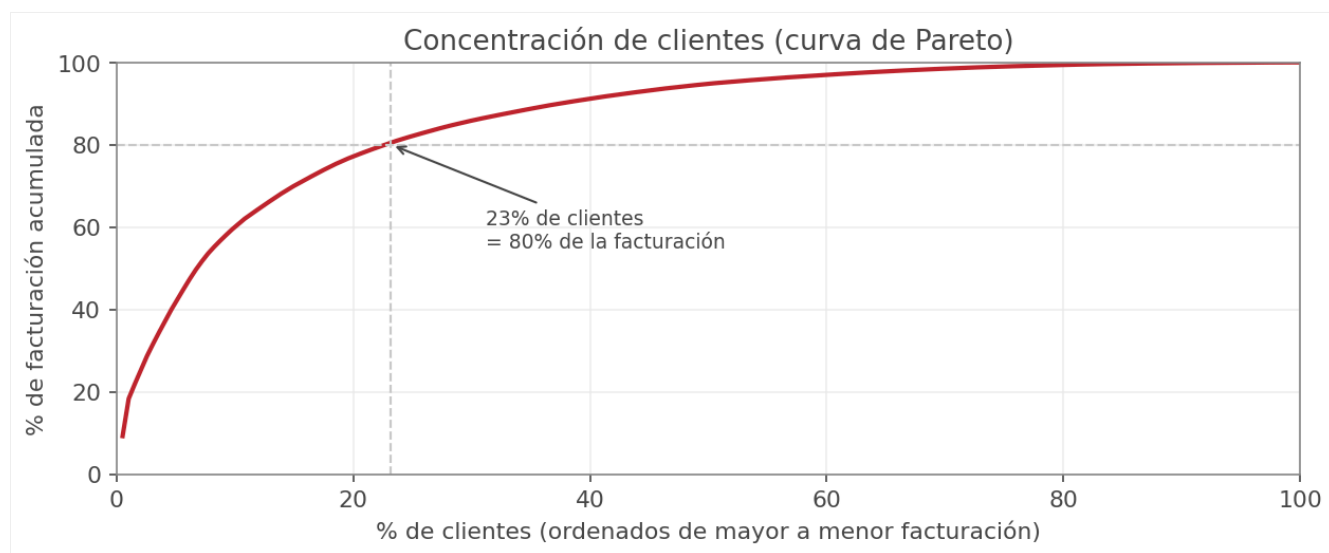


Figura 3. Curva de concentración de clientes por facturación acumulada.

Del lado de inventario, la distribución de lotes por meses restantes hasta vencer muestra que una porción del stock actual tiene una ventana de venta relativamente corta. Esa es la motivación directa del componente de reabastecimiento que prioriza la salida del inventario más próximo a vencer (sección 8):

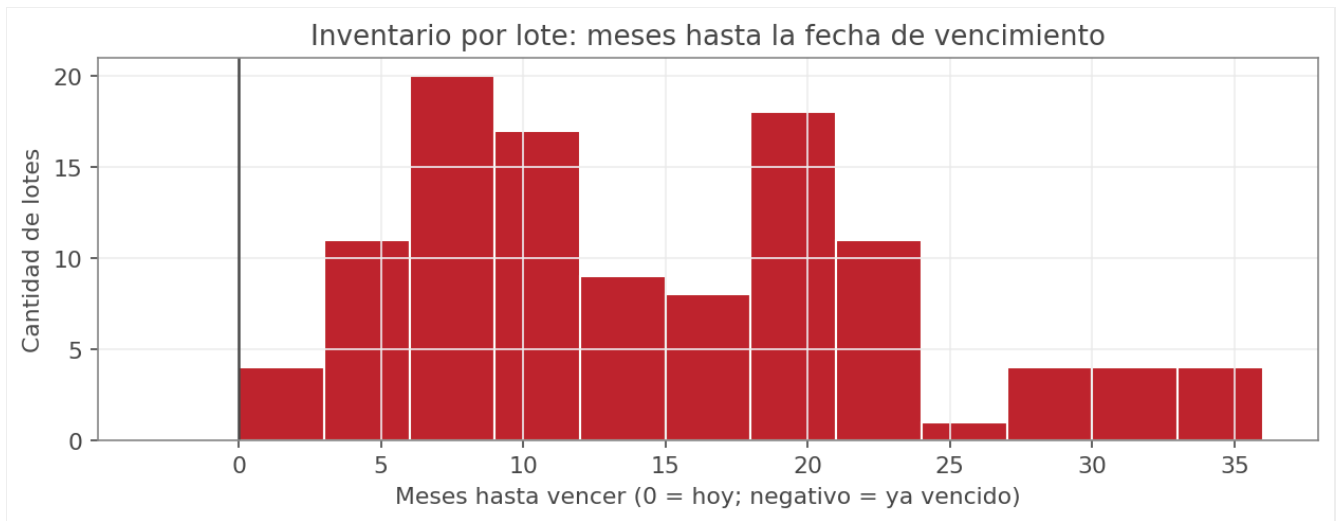


Figura 4. Distribución de lotes de inventario según meses restantes hasta la fecha de vencimiento.

7. Modelado predictivo de demanda

7.1 Por qué un modelo de dos etapas ("hurdle")

El panel producto $\times$ día es mayormente ceros: en el conjunto de validación de la versión vigente del modelo, solo el 5.7% de las combinaciones producto-día tuvieron venta real. Un único regresor entrenado directamente sobre esto tiende a "aprender a predecir cero" y subestimar los días en que sí hay demanda. Las primeras versiones del proyecto (v1-v4) usaron un solo LightGBM con objetivo Tweedie (pensado para variables con exceso de ceros); a partir de v5 se cambió a una arquitectura de dos etapas:

1. **Clasificación:** ¿hay venta ese producto ese día? (LightGBM binario, AUC = 0.90 en la versión vigente).
2. **Regresión:** dado que sí hay venta, ¿cuánto se vende? (LightGBM con objetivo Poisson, entrenado solo sobre filas con venta positiva).

La predicción final combina ambas: $E[\text{demanda}] = P(\text{venta}) \times E[\text{cantidad} \mid \text{venta}]$. Cada etapa aprende una sola pregunta en vez de las dos a la vez.

7.2 Variables (features)

Calendario (día del mes, semana del año, trimestre, fin de semana), eventos (días hasta/desde el evento más cercano), rezagos (1, 2, 3, 7, 14 y 28 días), medias y desviación móvil (7/28/90 días), e histórico promedio del producto. En total, 21 variables.

7.3 Validación y resultados

Split temporal (los últimos 150 días como validación, nunca aleatorio, para no filtrar información del futuro), y un backtest recursivo (el mismo procedimiento que usaría el modelo en producción) contra ventas ya ocurridas pero excluidas del entrenamiento:

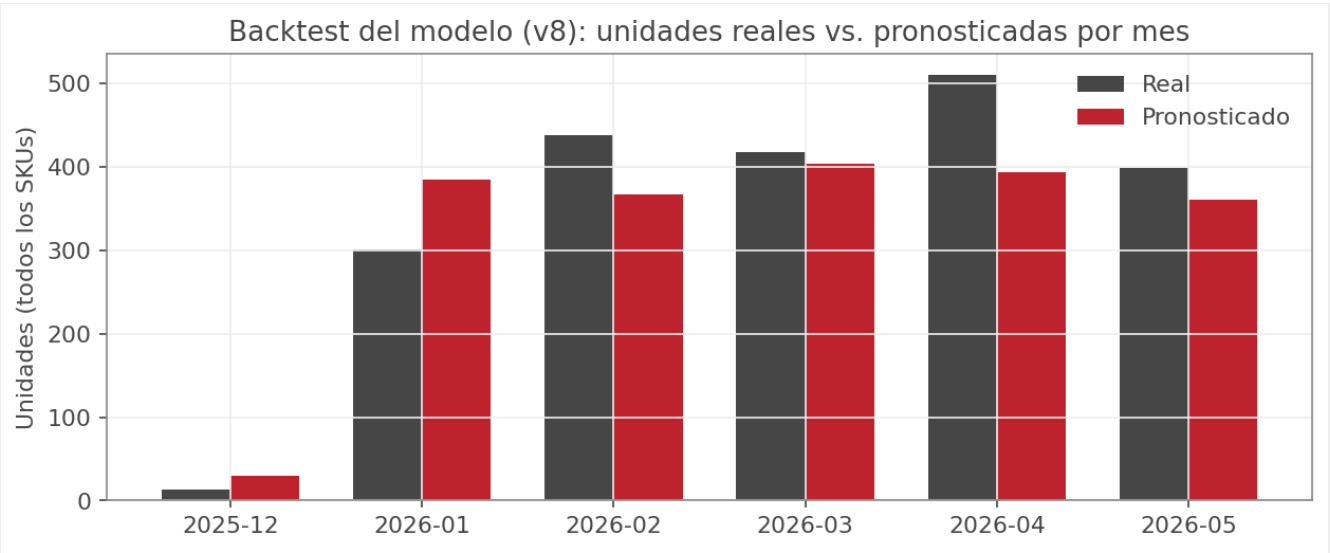


Figura 5. Backtest de la versión vigente (v8): unidades reales vs. pronosticadas, agregado mensual de todos los SKUs.

Métrica	Valor (v8)	Lectura de negocio
AUC de clasificación	0.90	Distingue bien los días con venta de los días sin venta
% del volumen total real (backtest)	93.4%	El volumen agregado pronosticado a 150 días se acerca mucho al real
% de combinaciones SKU-mes dentro de $\pm 20\%$	24.2%	Precisión a nivel granular, más exigente que el agregado

Se entrenaron y compararon 8 versiones del modelo durante el desarrollo, documentadas con sus métricas completas en el repositorio. La versión v8 mejora notablemente el error absoluto medio sobre las versiones anteriores, sin sacrificar la precisión agregada de volumen.

Tabla comparativa completa de las 8 versiones, metodología de features y discusión de por qué el WMAPE a nivel de fila es alto pese a que las métricas agregadas son buenas (naturaleza intermitente de la demanda), en `docs/modelado.md`.

8. Reabastecimiento: de la predicción a la decisión de compra

El pronóstico por sí solo no genera una decisión de compra: hace falta cruzarlo con el inventario existente (por lote, con su fecha de vencimiento), el lead time de fábrica, el stock de seguridad, y las restricciones reales de compra (cantidad mínima de pedido y múltiplo de empaque).

El componente de reabastecimiento simula el consumo del inventario existente en orden estricto de vencimiento (FIFO: lo que vence primero se consume primero) contra la **curva día a día** del pronóstico del modelo, no un promedio aplanado, para no perder la estacionalidad que el modelo sí calculó. Solo sugiere pedir la demanda que el inventario actual, más lo ya negociado con el proveedor, no alcanza a cubrir dentro del horizonte de compra.

Con los datos actuales, el modelo identifica ~495 unidades en riesgo real de vencer antes de consumirse al ritmo de venta pronosticado, y recomienda pedir 3.097 unidades adicionales entre los 91 SKUs (de 167) donde el inventario y lo ya comprometido no cubren la demanda esperada.

Detalle del algoritmo FIFO, la construcción del horizonte de simulación y la asignación de ofertas de proveedor, en `docs/arquitectura.md` y el código en `sponser_etl/replenishment/`.

9. Productivización: la aplicación web

Más allá del análisis, el proyecto se entrega como una aplicación en producción que el equipo comercial usa de forma continua. Esto cumple el criterio de que el modelo "reciba datos nuevos y devuelva una predicción" en un contexto real de negocio, no solo como notebook de análisis:

- **Carga de datos:** el equipo comercial sube los archivos de ventas/inventario actualizados directamente desde el navegador; el sistema valida el formato y dispara el pipeline completo (ETL + modelo + reabastecimiento) con un clic.
- **Dashboard interactivo:** KPIs de facturación vs. meta, catálogo con tendencia y riesgo de vencimiento, análisis comercial (ranking de clientes, Pareto, líneas comerciales), proyección por SKU, y pedido a fábrica con la recomendación ya calculada.
- **Seguridad y control de acceso:** autenticación por correo y contraseña con bloqueo tras intentos fallidos, y roles diferenciados (administrador vs. solo lectura) aplicados tanto en la interfaz como en cada endpoint del backend.
- **Gestión de colaboradores:** un administrador puede invitar a otros usuarios del equipo sin intervención técnica.

Capturas de pantalla, credenciales de acceso e instrucciones completas de instalación local en el README del repositorio.

10. Discusión: interpretabilidad y limitaciones

10.1 Interpretabilidad

Ambas etapas del modelo se auditaron por importancia de variable (ganancia de información acumulada en LightGBM). En la etapa de clasificación, el código de producto, la media móvil a 28 días y si el día es fin de semana son las variables más influyentes; en la de regresión, el código de producto y el rezago de 1 día dominan. Son resultados consistentes con la intuición de negocio: cada producto tiene su propio nivel base de demanda, y la venta reciente es el mejor predictor de la venta inmediata siguiente.

10.2 Limitaciones

- El error a nivel de fila individual (SKU-día) es alto por la naturaleza intermitente de la demanda (94% de días sin venta); las métricas agregadas (volumen total, SKU-mes) son las relevantes para la decisión de compra, no el pronóstico diario aislado.
- SKUs nuevos o con poca historia dependen más del promedio general del catálogo que de un patrón propio aprendido.
- El calendario de eventos deportivos que alimenta al modelo se mantiene manualmente y hoy tiene cobertura limitada. Su exactitud acota directamente cuánto puede aprender el modelo sobre el efecto de eventos en la demanda.
- La base de datos operativa (SQLite) es apropiada para el volumen actual de usuarios concurrentes; un despliegue con más usuarios simultáneos se beneficiaría de migrar a PostgreSQL.

11. Conclusiones y trabajo futuro

El proyecto demuestra que es posible pasar de reportes planos sin pronóstico a una herramienta productiva de decisión de compra, construida enteramente sobre datos reales de la empresa: un pipeline reproducible, un modelo de pronóstico con precisión agregada de negocio del 93.4% y una aplicación que el equipo comercial ya puede operar sin intervención técnica.

El mayor impacto de negocio identificado no es el pronóstico en sí, sino la combinación de pronóstico, inventario por lote y vencimiento: es lo que convierte una predicción estadística en una recomendación de compra concreta, con el riesgo de pérdida por vencimiento a la vista desde antes de que ocurra, no después.

Líneas de trabajo futuro:

- Poblar el calendario de eventos con datos reales (hoy es en su mayoría un ejemplo) para capturar mejor los picos de demanda ligados a carreras y promociones.
- Incorporar ofertas de proveedor reales al componente de reabastecimiento (`ofertas\_proveedor.csv` está vacío en el estado actual).
- Migrar la persistencia a PostgreSQL y el despliegue a un entorno cloud gestionado (Cloud Run + Cloud Storage), diseño ya evaluado pero pendiente de la infraestructura del cliente.
- Explorar técnicas de interpretabilidad más profundas (valores SHAP) más allá de la importancia de variable por ganancia.

12. Bibliografía y referencias

- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*. Advances in Neural Information Processing Systems (NeurIPS) 30.
- Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830.
- McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. Proceedings of the 9th Python in Science Conference.
- Croston, J. D. (1972). *Forecasting and Stock Control for Intermittent Demands*. Operational Research Quarterly, 23(3), 289-303. Referencia clásica sobre pronóstico de demanda intermitente, el mismo problema que motiva la arquitectura de dos etapas usada en este trabajo.
- Silver, E. A., Pyke, D. F., & Thomas, D. J. (2016). *Inventory and Production Management in Supply Chains* (4th ed.). CRC Press. Referencia general sobre lead time, stock de seguridad y políticas de reabastecimiento.
- FastAPI. Documentación oficial. <https://fastapi.tiangolo.com/> (consultado en 2026).

Nota: completar/ampliar esta bibliografía con las referencias específicas de los módulos del máster que respalden las decisiones metodológicas (arquitectura hurdle, validación temporal, gradient boosting) según el criterio del alumno.

13. Anexo: código fuente del pipeline

Este anexo no cuenta para el límite de extensión de la memoria.

Código fuente completo, tal como está en el repositorio (`sponser_etl/`), de las tres etapas descritas en las secciones 5, 7 y 8: extracción/transformación de datos, modelado de demanda y cálculo de reabastecimiento. El backend (`sponser_api/`) y el frontend no se incluyen aquí por extensión; están completos en el repositorio del proyecto.

`sponser_etl/extract.py`

Extracción de los .xls crudos de ventas/inventario

```
"""Extracción de los archivos crudos de ventas e inventario de Sponser.

Los archivos "COM Sponser *.xls" y "Control Sponser *.xls" no son binarios
Excel: son HTML (una tabla GridView de ASP.NET) exportado con extensión
.xls. Se parsean con html.parser (stdlib) para no depender de lxml/bs4.
"""

from __future__ import annotations

import glob
import os
import re
from dataclasses import dataclass, field
from html.parser import HTMLParser

import pandas as pd

# Columnas en el orden exacto en el que aparecen en el reporte
COLUMNAS_VENTAS = [
    "Tipo Documento", "nofactura", "Fecha", "Entidad", "Cliente",
    "No. Expediente", "telcel", "TipoReferencia", "tipocliente",
    "saldofactura", "Código", "item", "Proveedor", "Cantidad",
    "montounitario", "montocobrado", "descuentounitario", "descuentolinea",
    "Detalle", "montototal", "iv", "codigobarras", "Usuario", "Clave",
    "Moneda", "noreferencia", "tipopago",
]

class _GridViewTableParser(HTMLParser):
    """Extrae filas/celdas de la primera <table> del reporte GridView."""

    def __init__(self):
        super().__init__(convert_charrefs=True)
        self.rows: list[list[str]] = []
        self.in_table = False
        self.in_row = False
        self.in_cell = False
        self._current_row: list[str] = []
        self._current_text: list[str] = []
        self._table_done = False

    def handle_starttag(self, tag, attrs):
        if self._table_done:
```

```

        return
    if tag == "table" and not self._in_table:
        self._in_table = True
    elif self._in_table and tag == "tr":
        self._in_row = True
        self._current_row = []
    elif self._in_table and tag in ("td", "th"):
        self._in_cell = True
        self._current_text = []

    def handle_endtag(self, tag):
        if self._table_done:
            return
        if self._in_cell and tag in ("td", "th"):
            self._in_cell = False
            self._current_row.append("".join(self._current_text).strip())
        elif self._in_row and tag == "tr":
            self._in_row = False
            if self._current_row:
                self.rows.append(self._current_row)
        elif self._in_table and tag == "table":
            self._in_table = False
            self._table_done = True

    def handle_data(self, data):
        if self._in_cell:
            self._current_text.append(data)

def parse_html_xls(path: str) -> list[list[str]]:
    """Lee un archivo .xls (en realidad HTML) y devuelve filas de celdas."""
    with open(path, "r", encoding="utf-8", errors="replace") as f:
        contenido = f.read()
    parser = _GridViewTableParser()
    parser.feed(contenido)
    return parser.rows

@dataclass
class ResultadoExtraccion:
    ventas: pd.DataFrame
    errores: list[str] = field(default_factory=list)

_PATRON_NOMBRE = re.compile(r"^(COM|Control) Sponser (\d{4})\.xls$", re.IGNORECASE)

def _fuente_y_anio(nombre_archivo: str) -> tuple[str, int]:
    m = _PATRON_NOMBRE.match(nombre_archivo)
    if not m:
        raise ValueError(f"Nombre de archivo inesperado: {nombre_archivo}")
    fuente = "COM" if m.group(1).lower() == "com" else "Control"
    return fuente, int(m.group(2))

def extract_ventas(raw_dir: str) -> ResultadoExtraccion:
    """Extrae y concatena todos los archivos COM/Control Sponser *.xls de raw_dir."""
    patrones = [
        os.path.join(raw_dir, "COM Sponser *.xls"),
        os.path.join(raw_dir, "Control Sponser *.xls"),
    ]
    archivos = sorted(p for patron in patrones for p in glob.glob(patron))

    frames: list[pd.DataFrame] = []
    errores: list[str] = []

```

```

for path in archivos:
    nombre = os.path.basename(path)
    try:
        fuente, anio = _fuente_y_anio(nombre)
        filas = parse_html_xls(path)
        if not filas:
            errores.append(f"{nombre}: sin filas detectadas, se omite")
            continue

        header, datos = filas[0], filas[1:]
        if datos:
            datos = datos[:-1] # última fila = totales del reporte, no confiable

        # Por nombre, no por posición: reportes recientes traen una
        # columna "Marca" extra que los históricos no tienen.
        header_limpio = [h.strip() for h in header]
        faltantes = [c for c in COLUMNAS_VENTAS if c not in header_limpio]
        if faltantes:
            errores.append(
                f"{nombre}: faltan columnas esperadas {faltantes} (encabezado: {header_limpio}), se
omite"
            )
            continue

        df_crudo = pd.DataFrame(datos, columns=header_limpio)
        df = df_crudo[COLUMNAS_VENTAS] # por nombre: ignora columnas extra (ej. Marca)
        df["fuente"] = fuente
        df["anio"] = anio
        df["archivo_origen"] = nombre
        frames.append(df)
    except Exception as e: # noqa: BLE001 - no debe tumbar el pipeline por un archivo
        errores.append(f"{nombre}: error al procesar ({e})")

ventas = pd.concat(frames, ignore_index=True) if frames else pd.DataFrame(columns=COLUMNAS_VENTAS)
return ResultadoExtraccion(ventas=ventas, errores=errores)

def extract_inventario(raw_dir: str) -> pd.DataFrame:
    """Lee el archivo de inventario (xlsx genuino) de raw_dir."""
    candidatos = glob.glob(os.path.join(raw_dir, "Inventario*.xlsx"))
    if not candidatos:
        raise FileNotFoundError(f"No se encontró archivo Inventario*.xlsx en {raw_dir}")
    return pd.read_excel(candidatos[0])

```

sponser_etl/transform.py

Limpieza, normalización a USD y tablas agregadas

```

"""Limpieza, unificación y construcción de tablas agregadas.

Todas las tablas de salida quedan expresadas en dólares (USD): las pocas
filas en colones se convierten con TIPO_CAMBIO_USD antes de agregar nada.
"""

from __future__ import annotations

import pandas as pd

# Colones por dólar. Ajustar aquí si se necesita mayor precisión histórica;
# el impacto es mínimo (solo un puñado de filas del dataset están en colones).
TIPO_CAMBIO_USD = 530.0

```

```

COLUMNAS_MONETARIAS = [
    "saldo_factura", "monto_unitario", "monto_cobrado",
    "descuento_unitario", "descuento_linea", "monto_total", "iva",
]

_RENAME_VENTAS = {
    "Tipo Documento": "tipo_documento",
    "nofactura": "num_factura",
    "Fecha": "fecha",
    "Entidad": "entidad",
    "Cliente": "nombre_cliente",
    "No. Expediente": "id_cliente",
    "telcel": "telefono",
    "saldofactura": "saldo_factura",
    "Código": "codigo_interno_factura",
    "item": "item_descripcion",
    "Proveedor": "proveedor",
    "Cantidad": "cantidad",
    "montounitario": "monto_unitario",
    "montocobrado": "monto_cobrado",
    "descuentounitario": "descuento_unitario",
    "descuentolinea": "descuento_linea",
    "Detalle": "detalle",
    "montototal": "monto_total",
    "iv": "iva",
    "Usuario": "usuario",
    "Clave": "clave_hacienda",
    "Moneda": "moneda",
    "tipopago": "tipo_pago",
}

_COLUMNAS_DESCARTABLES = ["TipoReferencia", "tipocliente", "codigobarras", "noreferencia"]

# Líneas que no son venta de producto (p.ej. alquiler de inmueble/consultorio)
# y distorsionan el modelo de demanda/inventario: se excluyen del dataset.
_PATRONES_NO_PRODUCTO = [
    r"^\s*Servicios de alquiler",
    r"^\s*BUNDLE",
]

# Códigos de producto que se excluyen explícitamente de todo el pipeline (a
# pedido del negocio) -- no deben aparecer en ningún reporte ni alimentar el
# modelo de demanda/inventario.
_SKUS_EXCLUIDOS = {
    "1015", # Electrolytes Fruitmix 10 Tabs
    "1295", # Activator 200 Cola-Lemon UNIDAD 25ml
}

def _a_float(serie: pd.Series) -> pd.Series:
    limpia = serie.astype(str).str.replace(",", "", regex=False).str.strip()
    return pd.to_numeric(limpia, errors="coerce")

def limpiar_ventas(ventas_raw: pd.DataFrame) -> pd.DataFrame:
    if ventas_raw.empty:
        return ventas_raw

    df = ventas_raw.drop(columns=_COLUMNAS_DESCARTABLES, errors="ignore").rename(columns=_RENAME_VENTAS)

    for patron in _PATRONES_NO_PRODUCTO:
        excluir = df["item_descripcion"].astype(str).str.contains(patron, case=False, regex=True,
na=False)
        df = df[~excluir]

```

```

df = df.reset_index(drop=True)

df["fecha"] = pd.to_datetime(df["fecha"], dayfirst=True, errors="coerce")

for col in ["cantidad", *COLUMNAS_MONETARIAS]:
    df[col] = _a_float(df[col])

df["num_factura"] = pd.to_numeric(df["num_factura"], errors="coerce").astype("Int64")
df["id_cliente"] = pd.to_numeric(df["id_cliente"], errors="coerce").astype("Int64")
df["codigo_interno_factura"] = df["codigo_interno_factura"].astype(str).str.strip()
df["usuario"] = df["usuario"].astype(str).str.upper().str.strip()
df["nombre_cliente"] = df["nombre_cliente"].astype(str).str.strip()

# El código de producto real va embebido al inicio de "item"
# (p.ej. "17504 Activator 200 Fruit Boost..."), no en "Código"
# (ID interno de facturación, no cruza con inventario).
df["codigo_producto"] = df["item_descripcion"].astype(str).str.extract(r"^\s*(\d+)"[0]
df["codigo_producto"] = df["codigo_producto"].fillna(df["codigo_interno_factura"])

df = df[~df["codigo_producto"].isin(_SKUS_EXCLUIDOS)].reset_index(drop=True)

# Identificador único de factura: el correlativo se reinicia entre fuentes.
df["factura_id"] = df["fuente"] + "-" + df["num_factura"].astype(str)

# Conversión a USD: se normaliza moneda para que todo el análisis quede en una sola divisa.
df["moneda_original"] = df["moneda"].astype(str).str.strip()
es_colones = df["moneda_original"].str.upper().str.contains("COLON", na=False)
df.loc[es_colones, COLUMNAS_MONETARIAS] = df.loc[es_colones, COLUMNAS_MONETARIAS].div(TIPO_CAMBIO_USD)
df["moneda"] = "USD"

# "monto_total" incluye IVA; el pipeline trabaja en venta neta, así que
# se resta acá una sola vez. El bruto queda aparte para conciliación.
df["monto_total_bruto"] = df["monto_total"]
df["monto_total"] = (df["monto_total"] - df["iva"].fillna(0)).round(2)

return df

def separar_patrocinios(ventas: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
    """Todo cliente cuyo nombre contenga 'patrocinio' recibe producto como
    patrocinio deportivo -- no es una venta comercial y no debe contarse como
    tal (infla facturación/unidades y sesga el modelo de demanda). Se separa
    a su propio apartado en vez de simplemente descartarse."""
    if ventas.empty:
        return ventas, ventas
    es_patrocinio = ventas["nombre_cliente"].astype(str).str.contains("patrocinio", case=False, na=False)
    comerciales = ventas[~es_patrocinio].reset_index(drop=True)
    patrocinios = ventas[es_patrocinio].reset_index(drop=True)
    return comerciales, patrocinios

def construir_ventas_diarias_producto(ventas: pd.DataFrame) -> pd.DataFrame:
    if ventas.empty:
        return pd.DataFrame(columns=["fecha", "codigo_producto", "unidades_vendidas", "monto_total",
        "num_facturas"])

    agg = ventas.groupby(["fecha", "codigo_producto"]).agg(
        unidades_vendidas=("cantidad", "sum"),
        monto_total=("monto_total", "sum"),
        num_facturas=("factura_id", "nunique"),
    ).reset_index()
    return agg.sort_values(["fecha", "codigo_producto"]).reset_index(drop=True)

def construir_clientes_rfm(ventas: pd.DataFrame) -> pd.DataFrame:

```

```

if ventas.empty:
    return pd.DataFrame(columns=[
        "id_cliente", "nombre_cliente", "recencia_dias", "frecuencia",
        "monetario_total", "ticket_promedio", "productos_distintos",
        "antiguedad_dias", "fuente_principal",
    ])

fecha_max_global = ventas["fecha"].max()

nombres = (
    ventas.groupby("id_cliente")["nombre_cliente"]
    .agg(lambda s: s.mode().iat[0] if not s.mode().empty else s.iloc[0])
)
fuente_principal = (
    ventas.groupby("id_cliente")["fuente"]
    .agg(lambda s: s.mode().iat[0] if not s.mode().empty else s.iloc[0])
)

base = ventas.groupby("id_cliente").agg(
    fecha_min=("fecha", "min"),
    fecha_max=("fecha", "max"),
    frecuencia=("factura_id", "nunique"),
    monetario_total=("monto_total", "sum"),
    productos_distintos=("codigo_producto", "nunique"),
)

base["nombre_cliente"] = nombres
base["fuente_principal"] = fuente_principal
base["recencia_dias"] = (fecha_max_global - base["fecha_max"]).dt.days
base["antiguedad_dias"] = (base["fecha_max"] - base["fecha_min"]).dt.days
base["ticket_promedio"] = base["monetario_total"] / base["frecuencia"]

resultado = base.reset_index()[[
    "id_cliente", "nombre_cliente", "recencia_dias", "frecuencia",
    "monetario_total", "ticket_promedio", "productos_distintos",
    "antiguedad_dias", "fuente_principal",
]]
return resultado.sort_values("monetario_total", ascending=False).reset_index(drop=True)

def construir_inventario_lotes(inventario_raw: pd.DataFrame) -> pd.DataFrame:
    """Reshape wide->long: una fila por lote (Lote1 y Lote2 separados en el original)."""
    if inventario_raw.empty:
        return pd.DataFrame(columns=["codigo_producto", "descripcion", "lote_id", "cantidad",
    "fecha_vencimiento"])

    cols = list(inventario_raw.columns)
    # Posiciones fijas según el layout confirmado del reporte de inventario.
    col_codigo, col_desc = cols[0], cols[1]
    col_lote1_id, col_lote1_qty, col_lote1_venc = cols[4], cols[5], cols[6]
    col_lote2_id, col_lote2_qty, col_lote2_venc = cols[7], cols[8], cols[9]

    base = inventario_raw[[col_codigo, col_desc]].rename(
        columns={col_codigo: "codigo_producto", col_desc: "descripcion"}
    )
    base["codigo_producto"] = base["codigo_producto"].astype(str).str.strip()
    base["descripcion"] = base["descripcion"].astype(str).str.strip()

    lote1 = base.copy()
    lote1["lote_id"] = inventario_raw[col_lote1_id]
    lote1["cantidad"] = pd.to_numeric(inventario_raw[col_lote1_qty], errors="coerce")
    lote1["fecha_vencimiento"] = pd.to_datetime(inventario_raw[col_lote1_venc], errors="coerce")

    lote2 = base.copy()
    lote2["lote_id"] = inventario_raw[col_lote2_id]

```

```

lote2["cantidad"] = pd.to_numeric(inventario_raw[col_lote2_qty], errors="coerce")
lote2["fecha_vencimiento"] = pd.to_datetime(inventario_raw[col_lote2_venc], errors="coerce")

lotes = pd.concat([lote1, lote2], ignore_index=True)
lotes = lotes.dropna(subset=["cantidad"])
lotes = lotes[lotes["cantidad"] > 0].reset_index(drop=True)
return lotes[["codigo_producto", "descripcion", "lote_id", "cantidad", "fecha_vencimiento"]]

```

sponser_etl/load.py

Escritura de las tablas procesadas a CSV

```

"""Escritura de las tablas procesadas a CSV."""

from __future__ import annotations

import os

import pandas as pd

def guardar_tablas(tablas: dict[str, pd.DataFrame], out_dir: str) -> None:
    os.makedirs(out_dir, exist_ok=True)
    for nombre, df in tablas.items():
        ruta = os.path.join(out_dir, f"{nombre}.csv")
        df.to_csv(ruta, index=False, encoding="utf-8")

```

sponser_etl/pipeline.py

Orquestador del pipeline ETL

```

"""Orquestador del pipeline ETL de ventas Sponser.

Uso: python pipeline.py
Lee los .xls/.xlsx crudos de RAW_DIR, limpia y agrega los datos, y escribe
las tablas resultantes en data/processed/. Se puede volver a correr tal
cual cuando llegue el archivo de un nuevo año/periodo.
"""

from __future__ import annotations

import os
import sys

import pandas as pd

import extract
import transform
import load

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
RAW_DIR = os.path.join(BASE_DIR, "data", "raw")
OUT_DIR = os.path.join(BASE_DIR, "data", "processed")

def main() -> int:
    print(f"Leyendo archivos crudos desde: {RAW_DIR}")
    resultado = extract.extract_ventas(RAW_DIR)

```

```

for err in resultado.errores:
    print(f" [warning] {err}", file=sys.stderr)

if resultado.ventas.empty:
    print("No se extrajo ninguna fila de ventas. Revisa RAW_DIR y los nombres de archivo.")
    return 1

ventas_todo = transform.limpiar_ventas(resultado.ventas)
ventas, patrocinios = transform.separar_patrocinios(ventas_todo)
ventas_diarias = transform.construir_ventas_diarias_producto(ventas)
clientes_rfm = transform.construir_clientes_rfm(ventas)

try:
    inventario_raw = extract.extract_inventario(RAW_DIR)
    inventario_lotes = transform.construir_inventario_lotes(inventario_raw)
except FileNotFoundError as e:
    print(f" [warning] {e}", file=sys.stderr)
    inventario_lotes = transform.construir_inventario_lotes(pd.DataFrame())

tablas = {
    "ventas_detalle": ventas,
    "ventas_diarias_producto": ventas_diarias,
    "clientes_rfm": clientes_rfm,
    "inventario_lotes": inventario_lotes,
    "patrocinios": patrocinios,
}
load.guardar_tablas(tablas, OUT_DIR)

print("\nResumen:")
for nombre, df in tablas.items():
    print(f" {nombre}: {len(df):,} filas -> {os.path.join(OUT_DIR, nombre + '.csv')}")

if not ventas.empty:
    print(f"\n Rango de fechas en ventas: {ventas['fecha'].min().date()} -> {ventas['fecha'].max().date()}")
    print(f" Facturas distintas: {ventas['factura_id'].nunique():,}")
    print(f" Clientes distintos: {ventas['id_cliente'].nunique():,}")
    monedas_no_usd = (ventas['moneda'] != 'USD').sum()
    print(f" Filas con moneda distinta de USD tras conversión: {monedas_no_usd}")
if not patrocinios.empty:
    print(f"\n Patrocinios separados: {len(patrocinios):,} filas, "
          f"{patrocinios['id_cliente'].nunique():,} clientes, "
          f"${patrocinios['monto_total'].sum():,.2f} netos (excluidos de ventas)")

if resultado.errores:
    print(f"\n{len(resultado.errores)} advertencia(s) durante la extracción (ver arriba).")

return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

sponser_etl/modeling/features.py

Panel producto x fecha: calendario, eventos, rezagos, calibración

```

"""Construcción del panel producto x fecha con features de calendario, eventos y rezagos.

Compartido por train.py (panel histórico) y forecast.py (panel extendido día a día
durante el pronóstico recursivo). Es código único (no versionado por archivo) -
lo que varía entre versiones del modelo queda registrado en versions/vN/config.json.

```

```

"""

from __future__ import annotations

import json
import os

import numpy as np
import pandas as pd

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__))) # sponser_etl/
PROCESSED_DIR = os.path.join(BASE_DIR, "data", "processed")
EVENTOS_PATH = os.path.join(BASE_DIR, "eventos.csv")

LAGS = [1, 2, 3, 7, 14, 28]
ROLLING_WINDOWS = [7, 28, 90]

NUMERIC_FEATURES = [
    "dia_mes", "semana_anio", "trimestre", "es_fin_de_semana",
    "es_evento", "dias_hasta_proximo_evento", "dias_desde_evento_anterior",
    *[f"lag_{n}" for n in LAGS],
    *[f"rolling_mean_{n}" for n in ROLLING_WINDOWS],
    "rolling_std_7",
    "historico_promedio_producto",
]

CATEGORICAL_ENC_FEATURES = ["codigo_producto_enc", "dia_semana", "tipo_evento_activo_enc"]
MODEL_FEATURES = NUMERIC_FEATURES + CATEGORICAL_ENC_FEATURES
TARGET_COLUMN = "unidades_vendidas"

def cargarventas_diarias() -> pd.DataFrame:
    path = os.path.join(PROCESSED_DIR, "ventas_diarias_producto.csv")
    df = pd.read_csv(path, parse_dates=["fecha"], dtype={"codigo_producto": str})
    return df[["fecha", "codigo_producto", "unidades_vendidas"]].copy()

def cargar_eventos() -> pd.DataFrame:
    columnas = ["fecha_inicio", "fecha_fin", "tipo_evento", "nombre_evento"]
    if not os.path.exists(EVENTOS_PATH):
        return pd.DataFrame(columns=columnas)
    df = pd.read_csv(EVENTOS_PATH)
    if df.empty:
        return pd.DataFrame(columns=columnas)
    df["fecha_inicio"] = pd.to_datetime(df["fecha_inicio"], errors="coerce")
    df["fecha_fin"] = pd.to_datetime(df["fecha_fin"], errors="coerce")
    df = df.dropna(subset=["fecha_inicio", "fecha_fin"])
    df["tipo_evento"] = df["tipo_evento"].astype(str).str.strip().str.lower()
    return df

def construir_panel_base(ventas: pd.DataFrame, fecha_max: pd.Timestamp | None = None) -> pd.DataFrame:
    """Expande a grilla completa producto x fecha, rellenando ceros donde no hubo venta."""
    productos = ventas["codigo_producto"].unique()
    fecha_min = ventas["fecha"].min()
    fecha_max = fecha_max or ventas["fecha"].max()
    fechas = pd.date_range(fecha_min, fecha_max, freq="D")

    idx = pd.MultiIndex.from_product([productos, fechas], names=["codigo_producto", "fecha"])
    panel = pd.DataFrame(index=idx).reset_index()

    panel = panel.merge(ventas, on=["codigo_producto", "fecha"], how="left")
    panel["unidades_vendidas"] = panel["unidades_vendidas"].fillna(0.0)
    return panel.sort_values(["codigo_producto", "fecha"]).reset_index(drop=True)

```

```

def agregar_features_calendario(panel: pd.DataFrame) -> pd.DataFrame:
    panel["dia_semana"] = panel["fecha"].dt.dayofweek
    panel["mes"] = panel["fecha"].dt.month
    panel["dia_mes"] = panel["fecha"].dt.day
    panel["semana_anio"] = panel["fecha"].dt.isocalendar().week.astype(int)
    panel["trimestre"] = panel["fecha"].dt.quarter
    panel["es_fin_de_semana"] = panel["dia_semana"].isin([5, 6]).astype(int)
    return panel

def agregar_features_evento(panel: pd.DataFrame, eventos: pd.DataFrame) -> pd.DataFrame:
    fechas_unicas = panel["fecha"].drop_duplicates().sort_values().reset_index(drop=True)

    es_evento_por_fecha = {}
    tipo_por_fecha = {}
    for f in fechas_unicas:
        activos = eventos[(eventos["fecha_inicio"] <= f) & (eventos["fecha_fin"] >= f)]
        es_evento_por_fecha[f] = 1 if not activos.empty else 0
        tipo_por_fecha[f] = activos.iloc[0]["tipo_evento"] if not activos.empty else "ninguno"

    inicios = eventos["fecha_inicio"].sort_values().to_numpy() if not eventos.empty else np.array([],
dtype="datetime64[ns]")
    fines = eventos["fecha_fin"].sort_values().to_numpy() if not eventos.empty else np.array([],
dtype="datetime64[ns]")

    def _dias_hasta_proximo(f):
        futuros = inicios[inicios >= np.datetime64(f)]
        if len(futuros) == 0:
            return 999
        return int((futuros.min() - np.datetime64(f)) / np.timedelta64(1, "D"))

    def _dias_desde_anterior(f):
        pasados = fines[fines <= np.datetime64(f)]
        if len(pasados) == 0:
            return 999
        return int((np.datetime64(f) - pasados.max()) / np.timedelta64(1, "D"))

    dias_hasta = {f: _dias_hasta_proximo(f) for f in fechas_unicas}
    dias_desde = {f: _dias_desde_anterior(f) for f in fechas_unicas}

    panel["es_evento"] = panel["fecha"].map(es_evento_por_fecha).astype(int)
    panel["tipo_evento_activo"] = panel["fecha"].map(tipo_por_fecha)
    panel["dias_hasta_proximo_evento"] = panel["fecha"].map(dias_hasta).astype(int)
    panel["dias_desde_evento_anterior"] = panel["fecha"].map(dias_desde).astype(int)
    return panel

def agregar_features_rezago(panel: pd.DataFrame) -> pd.DataFrame:
    panel = panel.sort_values(["codigo_producto", "fecha"]).reset_index(drop=True)
    grouped = panel.groupby("codigo_producto")["unidades_vendidas"]
    for n in LAGS:
        panel[f"lag_{n}"] = grouped.shift(n)
    for n in ROLLING_WINDOWS:
        panel[f"rolling_mean_{n}"] = grouped.transform(lambda s, n=n: s.shift(1).rolling(n).mean())
    panel["rolling_std_7"] = grouped.transform(lambda s: s.shift(1).rolling(7).std())
    # Nivel histórico del producto (promedio expandido, sin ver el día actual): ancla
    # explícita para que el pronóstico recursivo no derive lejos del nivel real del
    # producto solo por depender de que el árbol separe bien 175 categorías.
    panel["historico_promedio_producto"] = grouped.transform(lambda s: s.shift(1).expanding().mean())
    return panel

def construir_panel(
    ventas: pd.DataFrame | None = None,
    eventos: pd.DataFrame | None = None,

```

```

    fecha_max: pd.Timestamp | None = None,
) -> pd.DataFrame:
    if ventas is None:
        ventas = cargar_ventas_diarias()
    if eventos is None:
        eventos = cargar_eventos()
    panel = construir_panel_base(ventas, fecha_max=fecha_max)
    panel = agregar_features_calendario(panel)
    panel = agregar_features_evento(panel, eventos)
    panel = agregar_features_rezago(panel)
    return panel

# --- Codificación de variables categóricas (consistente entre train y forecast) ---

def construir_encoders(panel: pd.DataFrame) -> dict:
    codigo_map = {c: i for i, c in enumerate(sorted(panel["codigo_producto"].unique()))}
    tipo_map = {t: i for i, t in enumerate(sorted(panel["tipo_evento_activo"].unique()))}
    if "ninguno" not in tipo_map:
        tipo_map["ninguno"] = len(tipo_map)
    return {"codigo_producto_map": codigo_map, "tipo_evento_map": tipo_map}

def aplicar_encoders(panel: pd.DataFrame, encoders: dict) -> pd.DataFrame:
    codigo_map = encoders["codigo_producto_map"]
    tipo_map = encoders["tipo_evento_map"]
    fallback_tipo = tipo_map.get("ninguno", -1)

    panel = panel.copy()
    panel["codigo_producto_enc"] = panel["codigo_producto"].map(codigo_map)
    panel["tipo_evento_activo_enc"] =
panel["tipo_evento_activo"].map(tipo_map).fillna(fallback_tipo).astype(int)
    return panel

def guardar_encoders(encoders: dict, path: str) -> None:
    with open(path, "w", encoding="utf-8") as f:
        json.dump(encoders, f, ensure_ascii=False, indent=2)

def cargar_encoders(path: str) -> dict:
    with open(path, "r", encoding="utf-8") as f:
        return json.load(f)

# --- Calibración por producto (post-modelo, se calcula una vez en train.py a
# partir de un backtest recursivo y se aplica igual en forecast.py) ---

def aplicar_calibracion(pred: np.ndarray, fila: pd.DataFrame, calibracion: dict | None) -> np.ndarray:
    """`fila` debe incluir la columna codigo_producto (no solo MODEL_FEATURES).
    Sin calibración, solo recorta a >= 0 (retrocompatible con versiones sin
    calibracion.json)."""
    pred = np.asarray(pred, dtype=float).copy()
    if not calibracion:
        return np.clip(pred, 0, None)

    inactivos = set(calibracion["productos_inactivos"])
    tercil_por_producto = calibracion["tercil_por_producto"]
    factores = calibracion["factores_por_tercil"]

    codigos = fila["codigo_producto"].astype(str).to_numpy()
    factor_por_fila = np.array([
        0.0 if c in inactivos else factores.get(tercil_por_producto.get(c, ""), 1.0)
        for c in codigos

```

```

    ])
    return np.clip(pred * factor_por_fila, 0, None)

def guardar_calibracion(calibracion: dict, path: str) -> None:
    with open(path, "w", encoding="utf-8") as f:
        json.dump(calibracion, f, ensure_ascii=False, indent=2)

def cargar_calibracion(path: str) -> dict | None:
    if not os.path.exists(path):
        return None
    with open(path, "r", encoding="utf-8") as f:
        return json.load(f)

```

sponser_etl/modeling/train.py

Entrenamiento del modelo hurdle (clasificación + regresión Poisson)

```

"""Entrena el modelo de demanda diaria por producto.

Uso: python train.py (desde sponser_etl/modeling/). Cada corrida crea una
versión nueva en versions/vN/ y actualiza versions/LATEST.txt. Split temporal:
últimos DIAS_VALIDACION días como validación, el resto entrenamiento.

TIPO_MODELO: "tweedie_directo" (un LightGBM prediciendo unidades directo,
usado en v1-v4) o "hurdle" (clasificador P(hay venta) + regresor Poisson de
cantidad dado que hubo venta; predicción final = P(venta) x E[cantidad|venta]).
"""

from __future__ import annotations

import datetime
import json
import os

import lightgbm as lgb
import numpy as np
import pandas as pd

import features
import forecast
import versioning

TIPO_MODELO = "hurdle"
DIAS_VALIDACION = 150 # debe coincidir con DIAS_BACKTEST en backtest.py
DIAS_ACTIVIDAD_RECIENTE = 90 # sin venta en esta ventana antes del corte -> se considera inactivo

HYPERPARAMS_COMUNES = dict(
    n_estimators=1500,
    learning_rate=0.03,
    num_leaves=15,
    min_child_samples=50,
    reg_lambda=1.0,
    random_state=42,
)

HYPERPARAMS_TWEEDIE = dict(objective="tweedie", tweedie_variance_power=1.1, **HYPERPARAMS_COMUNES)
HYPERPARAMS_CLASIFICACION = dict(objective="binary", **HYPERPARAMS_COMUNES)
HYPERPARAMS_REGRESION_POSITIVOS = dict(objective="poisson", **HYPERPARAMS_COMUNES)

def wmape(y_true: np.ndarray, y_pred: np.ndarray) -> float:

```

```

return float(np.abs(y_true - y_pred).sum() / max(np.abs(y_true).sum(), 1e-9))

def calibrar_via_backtest_recursivo(
    modelo_crudo: dict,
    encoders: dict,
    ventas: pd.DataFrame,
    cutoff: pd.Timestamp,
    fecha_max: pd.Timestamp,
    dias: int,
    dias_actividad_reciente: int = DIAS_ACTIVIDAD_RECIENTE,
    n_terciles: int = 3,
) -> dict:
    """Calibración por producto, corriendo el mismo pronóstico recursivo de
    forecast.py/backtest.py sobre la ventana de validación (modelo sin
    calibrar) en vez de usar métricas de un solo paso."""
    historial_train = ventas[ventas["fecha"] < cutoff].copy()
    print(f"\nCalculando calibración con un backtest recursivo interno "
          f"({dias} días, {cutoff.date()} -> {fecha_max.date()}, modelo sin calibrar)...")
    pred_df = forecast.forecast_recursivo(modelo_crudo, encoders, dias=dias,
    historial_inicial=historial_train)

    productos = historial_train["codigo_producto"].unique()
    fechas_val = pd.date_range(cutoff, fecha_max, freq="D")
    idx = pd.MultiIndex.from_product([productos, fechas_val], names=["codigo_producto", "fecha"])
    real_df = pd.DataFrame(index=idx).reset_index().merge(ventas, on=["codigo_producto", "fecha"],
    how="left")
    real_df["unidades_vendidas"] = real_df["unidades_vendidas"].fillna(0.0)
    comp = pred_df.merge(real_df, on=["fecha", "codigo_producto"], how="inner")

    por_producto = comp.groupby("codigo_producto").agg(
        real=("unidades_vendidas", "sum"),
        pronosticado=("unidades_pronosticadas", "sum"),
    )

    venta_reciente = historial_train[historial_train["fecha"] >= cutoff -
    pd.Timedelta(days=dias_actividad_reciente)]
    venta_reciente_por_producto = venta_reciente.groupby("codigo_producto")["unidades_vendidas"].sum()
    por_producto["venta_reciente"] = por_producto.index.map(venta_reciente_por_producto).fillna(0.0)

    productos_inactivos = por_producto.index[por_producto["venta_reciente"] <= 0].tolist()

    activos = por_producto[por_producto["venta_reciente"] > 0].copy()
    promedio_historico = historial_train.groupby("codigo_producto")["unidades_vendidas"].mean()
    activos["promedio_historico"] = activos.index.map(promedio_historico).fillna(0.0)

    tercil_por_producto: dict[str, str] = {}
    factores_por_tercil: dict[str, float] = {}
    if len(activos) >= n_terciles:
        terciles = pd.qcut(activos["promedio_historico"], n_terciles, duplicates="drop")
        for etiqueta, grupo in activos.groupby(terciles, observed=True):
            etiqueta = str(etiqueta)
            real_sum = grupo["real"].sum()
            pred_sum = grupo["pronosticado"].sum()
            factores_por_tercil[etiqueta] = float(real_sum / pred_sum) if pred_sum > 1e-9 else 1.0
            for cod in grupo.index:
                tercil_por_producto[cod] = etiqueta

    return {
        "productos_inactivos": productos_inactivos,
        "tercil_por_producto": tercil_por_producto,
        "factores_por_tercil": factores_por_tercil,
        "dias_actividad_reciente": dias_actividad_reciente,
        "n_productos_inactivos": len(productos_inactivos),
    }

```

```

def entrenar_tweedie_directo(X_train, y_train, X_val, y_val):
    modelo = lgb.LGBMRegressor(**HYPERPARAMS_TWEEDIE)
    modelo.fit(
        X_train, y_train,
        eval_set=[(X_val, y_val)],
        eval_metric="mae",
        categorical_feature=features.CATEGORICAL_ENC_FEATURES,
        callbacks=[lgb.early_stopping(stopping_rounds=80), lgb.log_evaluation(period=100)],
    )
    pred_val = np.clip(modelo.predict(X_val), 0, None)
    return {
        "tipo": "tweedie_directo",
        "boosters": {"regresion": modelo.booster_},
        "hyperparams": {"regresion": HYPERPARAMS_TWEEDIE},
        "pred_val": pred_val,
    }

def entrenar_hurdle(X_train, y_train, X_val, y_val):
    y_train_bin = (y_train > 0).astype(int)
    y_val_bin = (y_val > 0).astype(int)

    print("\n--- Etapa 1: clasificación (¿hay venta ese producto ese día?) ---")
    clasificador = lgb.LGBMClassifier(**HYPERPARAMS_CLASIFICACION)
    clasificador.fit(
        X_train, y_train_bin,
        eval_set=[(X_val, y_val_bin)],
        eval_metric="auc",
        categorical_feature=features.CATEGORICAL_ENC_FEATURES,
        callbacks=[lgb.early_stopping(stopping_rounds=80), lgb.log_evaluation(period=100)],
    )
    p_val = clasificador.predict_proba(X_val)[:, 1]

    print("\n--- Etapa 2: regresión (cuánto se vende, dado que sí hubo venta) ---")
    mask_train_pos = y_train > 0
    mask_val_pos = y_val > 0
    regresor = lgb.LGBMRegressor(**HYPERPARAMS_REGRESION_POSITIVOS)
    regresor.fit(
        X_train[mask_train_pos], y_train[mask_train_pos],
        eval_set=[(X_val[mask_val_pos], y_val[mask_val_pos])],
        eval_metric="mae",
        categorical_feature=features.CATEGORICAL_ENC_FEATURES,
        callbacks=[lgb.early_stopping(stopping_rounds=80), lgb.log_evaluation(period=100)],
    )
    q_val = regresor.predict(X_val)

    pred_val = np.clip(p_val * q_val, 0, None)

    return {
        "tipo": "hurdle",
        "boosters": {"clasificacion": clasificador.booster_, "regresion": regresor.booster_},
        "hyperparams": {"clasificacion": HYPERPARAMS_CLASIFICACION, "regresion":
HYPERPARAMS_REGRESION_POSITIVOS},
        "pred_val": pred_val,
        "diagnostico": {
            "auc_clasificacion": float(clasificador.booster_.best_score["valid_0"]["auc"]),
            "tasa_venta_real_val": float(y_val_bin.mean()),
        },
    }

def main() -> int:
    version = versioning.siguiente_version()

```

```

vdir = versioning.version_dir(version)
print(f"Entrenando versión {version} (tipo: {TIPO_MODELO}) -> {vdir}")

print("Construyendo panel histórico (producto x fecha, calendario, eventos, rezagos)...")
panel = features.construir_panel()

encoders = features.construir_encoders(panel)
panel = features.aplicar_encoders(panel, encoders)
features.guardar_encoders(encoders, os.path.join(vdir, "encoders.json"))

fecha_max = panel["fecha"].max()
cutoff = fecha_max - pd.Timedelta(days=DIAS_VALIDACION - 1)
train_df = panel[panel["fecha"] < cutoff]
val_df = panel[panel["fecha"] >= cutoff]

print(f"Entrenamiento: {len(train_df):,} filas ({panel['fecha'].min().date() -> {(cutoff -
pd.Timedelta(days=1)).date()})")
print(f"Validación: {len(val_df):,} filas ({cutoff.date() -> {fecha_max.date()})")

X_train, y_train = train_df[features.MODEL_FEATURES], train_df[features.TARGET_COLUMN]
X_val, y_val = val_df[features.MODEL_FEATURES], val_df[features.TARGET_COLUMN]

if TIPO_MODELO == "hurdle":
    resultado = entrenar_hurdle(X_train, y_train, X_val, y_val)
elif TIPO_MODELO == "tweedie_directo":
    resultado = entrenar_tweedie_directo(X_train, y_train, X_val, y_val)
else:
    raise ValueError(f"TIPO_MODELO desconocido: {TIPO_MODELO}")

pred_val_crudo = resultado["pred_val"]
mae_crudo = float(np.mean(np.abs(y_val.to_numpy() - pred_val_crudo)))
w_crudo = wmape(y_val.to_numpy(), pred_val_crudo)

modelo_crudo = {"tipo": resultado["tipo"], **resultado["boosters"], "calibracion": None}
ventas_completas = features.cargar_ventas_diarias()
calibracion = calibrar_via_backtest_recursivo(
    modelo_crudo, encoders, ventas_completas, cutoff, fecha_max, DIAS_VALIDACION
)
features.guardar_calibracion(calibracion, os.path.join(vdir, "calibracion.json"))

pred_val = features.aplicar_calibracion(pred_val_crudo, val_df, calibracion)
mae = float(np.mean(np.abs(y_val.to_numpy() - pred_val)))
rmse = float(np.sqrt(np.mean((y_val.to_numpy() - pred_val) ** 2)))
w = wmape(y_val.to_numpy(), pred_val)

print(f"\nFactores de calibración por tercil: {calibracion['factores_por_tercil']}")
print(f"Productos sin venta en los últimos {DIAS_ACTIVIDAD_RECIENTE} días antes del corte "
      f"(forzados a 0): {calibracion['n_productos_inactivos']}")

print(f"\nMétricas de validación single-step ({TIPO_MODELO}, con rezagos reales - la calibración "
      f"se calculó recursivamente pero acá se aplica también a este set de un solo paso, solo "
      f"como referencia rápida; la comparación real está en backtest.py):")
print(f" Sin calibrar -> MAE: {mae_crudo:.3f} WMAPE: {w_crudo:.1%}")
print(f" Calibrado -> MAE: {mae:.3f} RMSE: {rmse:.3f} WMAPE: {w:.1%}")
if "diagnostico" in resultado:
    d = resultado["diagnostico"]
    print(f" AUC clasificación (¿hay venta?): {d['auc_clasificacion']:.3f}")
    print(f" Tasa real de días con venta en validación: {d['tasa_venta_real_val']:.1%}")

model_paths = {}
for nombre, booster in resultado["boosters"].items():
    path = os.path.join(vdir, f"model_{nombre}.txt")
    booster.save_model(path)
    model_paths[nombre] = os.path.basename(path)
    print(f"Modelo '{nombre}' guardado en: {path}")

```

```

importancias = pd.Series(
    booster.feature_importance(importance_type="gain"),
    index=booster.feature_name(),
).sort_values(ascending=False)
print(f"\nTop 10 features por importancia ({nombre}, gain):")
print(importancias.head(10).to_string())

config = {
    "version": version,
    "tipo_modelo": TIPO_MODELO,
    "fecha_entrenamiento": datetime.datetime.now().isoformat(timespec="seconds"),
    "dias_validacion": DIAS_VALIDACION,
    "lags": features.LAGS,
    "rolling_windows": features.ROLLING_WINDOWS,
    "model_features": features.MODEL_FEATURES,
    "hyperparams": resultado["hyperparams"],
    "model_files": model_paths,
}

with open(os.path.join(vdir, "config.json"), "w", encoding="utf-8") as f:
    json.dump(config, f, ensure_ascii=False, indent=2)

metrics = {"mae_validacion": mae, "rmse_validacion": rmse, "wmape_validacion": w}
if "diagnostico" in resultado:
    metrics.update(resultado["diagnostico"])
with open(os.path.join(vdir, "metrics.json"), "w", encoding="utf-8") as f:
    json.dump(metrics, f, ensure_ascii=False, indent=2)

versioning.actualizar_resumen(version, {
    "tipo_modelo": TIPO_MODELO,
    "fecha": config["fecha_entrenamiento"],
    "dias_validacion": DIAS_VALIDACION,
    "n_features": len(features.MODEL_FEATURES),
    "mae_validacion": round(mae, 4),
    "wmape_validacion": round(w, 4),
})

versioning.marcar_latest(version)
print(f"\nVersión {version} lista y marcada como LATEST.")

return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

sponser_etl/modeling/forecast.py

Pronóstico recursivo día a día a partir de una versión entrenada

"""Pronóstico recursivo de demanda a 100 días por producto, usando una versión entrenada del modelo (por defecto, la más reciente en versions/LATEST.txt).

Uso: python forecast.py (después de correr train.py al menos una vez).
 Para cada día futuro (a partir del último día con datos reales), calcula features de calendario/evento/rezago usando el historial real + lo ya pronosticado, predice con el modelo guardado, y encadena el resultado como insumo del siguiente día.

"""

```
from __future__ import annotations
```

```

import json
import os
import sys

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__)))) # sponser_etl/ (para load.py)

import lightgbm as lgb
import numpy as np
import pandas as pd

import features
import load
import versioning

OUT_DIR = features.PROCESSED_DIR
HORIZONTE_DIAS = 100

def predecir(modelo: dict, fila: pd.DataFrame) -> np.ndarray:
    """Predicción combinada, sin importar si `modelo` es hurdle o tweedie_directo,
    con la calibración por producto ya aplicada (si existe). `fila` debe incluir
    codigo_producto además de MODEL_FEATURES (la calibración es por producto)."""
    X = fila[features.MODEL_FEATURES]
    if modelo["tipo"] == "hurdle":
        p = modelo["clasificacion"].predict(X)
        q = modelo["regresion"].predict(X)
        pred = p * q
    else:
        pred = modelo["regresion"].predict(X)
    return features.aplicar_calibracion(pred, fila, modelo.get("calibracion"))

def forecast_recursoivo(
    modelo: dict,
    encoders: dict,
    dias: int = HORIZONTE_DIAS,
    historial_inicial: pd.DataFrame | None = None,
) -> pd.DataFrame:
    """Pronostica `dias` dias recursivamente a partir del último día del historial.

    Por defecto usa todo el historial real disponible (pronóstico "hacia el futuro").
    Se puede pasar `historial_inicial` truncado a una fecha de corte anterior para
    hacer un backtest: el modelo pronostica recursivamente sin ver los datos reales
    posteriores al corte, igual que pronosticaría hacia un futuro genuino.
    """
    historial = historial_inicial if historial_inicial is not None else features.cargar_ventas_diarias()
    eventos = features.cargar_eventos()
    ultimo_dia_real = historial["fecha"].max()

    predicciones = []
    for i in range(1, dias + 1):
        fecha_objetivo = ultimo_dia_real + pd.Timedelta(days=i)
        panel = features.construir_panel(ventas=historial, eventos=eventos, fecha_max=fecha_objetivo)
        panel = features.aplicar_encoders(panel, encoders)

        fila = panel[panel["fecha"] == fecha_objetivo].copy()
        pred = predecir(modelo, fila)
        fila["unidades_vendidas"] = pred

        predicciones.append(fila[["fecha", "codigo_producto", "unidades_vendidas"]].rename(
            columns={"unidades_vendidas": "unidades_pronosticadas"}
        ))
    historial = pd.concat(
        [historial, fila[["fecha", "codigo_producto", "unidades_vendidas"]]],

```

```

        ignore_index=True,
    )

    if i % 10 == 0 or i == dias:
        print(f"  pronosticado día {i}/{dias} ({fecha_objetivo.date()})")

    return pd.concat(predicciones, ignore_index=True)

def cargar_modelo(version: str | None = None):
    """Carga la versión pedida (o LATEST) y devuelve (version, modelo, encoders).

    `modelo` es un dict {"tipo": ..., "regresion": Booster} para tweedie_directo,
    o {"tipo": "hurdle", "clasificacion": Booster, "regresion": Booster} para hurdle
    - soporta versiones antiguas (v1-v4, sin config.json con tipo_modelo) asumiendo
    tweedie_directo con model_lightgbm.txt.
    """
    version = versioning.resolver_version(version)
    vdir = versioning.version_dir(version)

    config_path = os.path.join(vdir, "config.json")
    tipo = "tweedie_directo"
    if os.path.exists(config_path):
        with open(config_path, "r", encoding="utf-8") as f:
            tipo = json.load(f).get("tipo_modelo", "tweedie_directo")

    encoders = features.cargar_encoders(os.path.join(vdir, "encoders.json"))

    if tipo == "hurdle":
        clas_path = os.path.join(vdir, "model_clasificacion.txt")
        reg_path = os.path.join(vdir, "model_regresion.txt")
        if not (os.path.exists(clas_path) and os.path.exists(reg_path)):
            raise FileNotFoundError(f"Faltan archivos de modelo hurdle de la versión {version} en {vdir}.")
        modelo = {
            "tipo": "hurdle",
            "clasificacion": lgb.Booster(model_file=clas_path),
            "regresion": lgb.Booster(model_file=reg_path),
        }
    else:
        model_path = os.path.join(vdir, "model_lightgbm.txt")
        if not os.path.exists(model_path):
            raise FileNotFoundError(f"No se encontró el modelo de la versión {version} en {model_path}.")
        modelo = {"tipo": "tweedie_directo", "regresion": lgb.Booster(model_file=model_path)}

    modelo["calibracion"] = features.cargar_calibracion(os.path.join(vdir, "calibracion.json"))

    return version, modelo, encoders

def main() -> int:
    version, modelo, encoders = cargar_modelo()
    print(f"Usando versión: {version} (tipo: {modelo['tipo']})")

    print(f"Pronosticando {HORIZONTE_DIAS} días hacia adelante...")
    forecast_df = forecast_recursoivo(modelo, encoders, dias=HORIZONTE_DIAS)

    resumen_sku = (
        forecast_df.groupby("codigo_producto")["unidades_pronosticadas"]
        .mean()
        .reset_index()
        .rename(columns={"unidades_pronosticadas": "daily_demand"})
        .sort_values("daily_demand", ascending=False)
    )

```

```

load.guardar_tablas(
    {
        "forecast_demanda_producto": forecast_df,
        "demanda_promedio_sku": resumen_sku,
    },
    OUT_DIR,
)

print(f"\nforecast_demanda_producto.csv: {len(forecast_df):,} filas -> {OUT_DIR}")
print(f"demanda_promedio_sku.csv: {len(resumen_sku):,} filas -> {OUT_DIR}")
print(f"Rango pronosticado: {forecast_df['fecha'].min().date()} ->
{forecast_df['fecha'].max().date()}")
print(f"(generado con la versión del modelo: {version})")

return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

sponser_etl/modeling/backtest.py

Validación del pronóstico recursivo contra venta real ya ocurrida

"""Backtest: mide qué tan preciso es el pronóstico recursivo comparándolo contra ventas reales que ya ocurrieron (no contra el futuro).

Usa una versión ya entrenada (por defecto la más reciente) – train.py la entrenó excluyendo los últimos DIAS_BACKTEST días reales – y corre exactamente el mismo procedimiento recursivo de forecast.py, pero apuntado a ese periodo que ya conocemos, para comparar predicción vs. venta real día por día, mes por mes y producto por producto. Los resultados quedan guardados dentro de la carpeta de la versión evaluada (versions/vN/), y el resumen global en versions/summary.csv.

Uso: python backtest.py (después de correr train.py)

```

"""
from __future__ import annotations

import os

import lightgbm as lgb
import numpy as np
import pandas as pd

import features
import forecast
import versioning

DIAS_BACKTEST = 150 # debe coincidir con DIAS_VALIDACION en train.py

def wmape(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    return float(np.abs(y_true - y_pred).sum() / max(np.abs(y_true).sum(), 1e-9))

def cargar_descripciones() -> pd.Series:
    """codigo_producto -> descripción más frecuente en ventas_detalle.csv."""
    ventas = pd.read_csv(
        os.path.join(features.PROCESSED_DIR, "ventas_detalle.csv"),
        dtype={"codigo_producto": str},
        usecols=["codigo_producto", "item_descripcion"],

```

```

)
ventas["descripcion"] = ventas["item_descripcion"].astype(str).str.replace(
    r"^\s*\d+\s*", "", regex=True
).str.strip()
return ventas.groupby("codigo_producto")["descripcion"].agg(lambda s: s.mode().iat[0])

def main() -> int:
    version, modelo, encoders = forecast.cargar_modelo()
    vdir = versioning.version_dir(version)
    print(f"Backtesteando versión: {version} (tipo: {modelo['tipo']})")

    ventas = features.cargar_ventas_diarias()
    fecha_max = ventas["fecha"].max()
    cutoff = fecha_max - pd.Timedelta(days=DIAS_BACKTEST - 1)

    historial_train = ventas[ventas["fecha"] < cutoff].copy()
    print(f"Pronosticando recursivamente {DIAS_BACKTEST} días desde {cutoff.date()} "
          f"hasta {fecha_max.date()} (la versión {version} NO vio datos reales de ese período al "
          f"entrenar)...")

    pred_df = forecast.forecast_recursivo(
        modelo, encoders, dias=DIAS_BACKTEST, historial_inicial=historial_train
    )

    productos = historial_train["codigo_producto"].unique()
    fechas_val = pd.date_range(cutoff, fecha_max, freq="D")
    idx = pd.MultiIndex.from_product([productos, fechas_val], names=["codigo_producto", "fecha"])
    real_df = pd.DataFrame(index=idx).reset_index().merge(ventas, on=["codigo_producto", "fecha"],
how="left")
    real_df["unidades_vendidas"] = real_df["unidades_vendidas"].fillna(0.0)

    comp = pred_df.merge(real_df, on=["fecha", "codigo_producto"], how="inner")
    comp["error"] = comp["unidades_pronosticadas"] - comp["unidades_vendidas"]
    comp["dia_horizonte"] = (comp["fecha"] - cutoff).dt.days + 1

    mae = comp["error"].abs().mean()
    rmse = np.sqrt((comp["error"] ** 2).mean())
    w = wmape(comp["unidades_vendidas"].to_numpy(), comp["unidades_pronosticadas"].to_numpy())
    sesgo = comp["error"].mean()
    total_real = comp["unidades_vendidas"].sum()
    total_pred = comp["unidades_pronosticadas"].sum()

    print(f"\n=== Backtest recursivo ({DIAS_BACKTEST} días, comparado contra venta real) ===")
    print(f"  Filas comparadas: {len(comp):,} (producto x día)")
    print(f"  MAE: {mae:.3f} unidades/producto/día")
    print(f"  RMSE: {rmse:.3f}")
    print(f"  WMAPE: {w:.1%}")
    print(f"  Sesgo promedio: {sesgo:+.3f} unidades/producto/día ({'SUBESTIMA' if sesgo < 0 else 'SOBRESTIMA'})")
    print(f"  Total real en el período: {total_real:.0f} unidades")
    print(f"  Total pronosticado en el período: {total_pred:.0f} unidades ({total_pred / total_real:.1%} del real)")

    print("\n--- Precisión por tramo del horizonte (agrupado en rangos de días; un solo día suelto puede\n"
          "      tener venta real ~0 entre 175 productos y desestabilizar el WMAPE) ---")
    tramos = [(1, 7), (8, 14), (15, 30), (31, 60), (61, 90), (91, 120), (121, 150)]
    tramos = [t for t in tramos if t[0] <= DIAS_BACKTEST]
    comp["tramo"] = pd.cut(
        comp["dia_horizonte"],
        bins=[t[0] - 1 for t in tramos] + [min(tramos[-1][1], DIAS_BACKTEST)],
        labels=[f"días {t[0]}-{min(t[1], DIAS_BACKTEST)}" for t in tramos],
    )
    por_tramo = comp.groupby("tramo", observed=True).apply(

```

```

        lambda g: pd.Series({
            "real": g["unidades_vendidas"].sum(),
            "pronosticado": g["unidades_pronosticadas"].sum(),
            "wmape": wmape(g["unidades_vendidas"].to_numpy(), g["unidades_pronosticadas"].to_numpy()),
        })),
        include_groups=False,
    )
print(por_tramo.to_string(float_format=lambda x: f"{x:.3f}"))

print("\n--- Top 10 productos de mayor venta real: real vs pronosticado (total del período) ---")
por_producto = comp.groupby("codigo_producto").agg(
    real=("unidades_vendidas", "sum"),
    pronosticado=("unidades_pronosticadas", "sum"),
)
por_producto["error_pct"] = (
    (por_producto["pronosticado"] - por_producto["real"]) / por_producto["real"].replace(0, np.nan)
)
top10 = por_producto.sort_values("real", ascending=False).head(10)
print(top10.to_string(float_format=lambda x: f"{x:.2f}"))

por_producto.reset_index().sort_values("real", ascending=False).to_csv(
    os.path.join(vdir, "backtest_resultados.csv"), index=False
)

# --- Agregación mensual ---
comp["mes"] = comp["fecha"].dt.to_period("M").astype(str)
descripciones = cargar_descripciones()

mensual_producto = comp.groupby(["codigo_producto", "mes"]).agg(
    real=("unidades_vendidas", "sum"),
    pronosticado=("unidades_pronosticadas", "sum"),
).reset_index()
mensual_producto["descripcion"] = mensual_producto["codigo_producto"].map(descripciones).fillna("")
mensual_producto["error_pct"] = (
    (mensual_producto["pronosticado"] - mensual_producto["real"]) /
mensual_producto["real"].replace(0, np.nan)
)
mensual_producto = mensual_producto[
    ["codigo_producto", "descripcion", "mes", "real", "pronosticado", "error_pct"]
].sort_values(["mes", "real"], ascending=[True, False])

mensual_total = comp.groupby("mes").apply(
    lambda g: pd.Series({
        "real": g["unidades_vendidas"].sum(),
        "pronosticado": g["unidades_pronosticadas"].sum(),
        "wmape": wmape(g["unidades_vendidas"].to_numpy(), g["unidades_pronosticadas"].to_numpy()),
    })),
    include_groups=False,
).reset_index()
mensual_total["error_pct"] = (mensual_total["pronosticado"] - mensual_total["real"]) /
mensual_total["real"]

print("\n=== Exactitud mensual, total compañía ===")
print(mensual_total.to_string(index=False, float_format=lambda x: f"{x:.3f}"))

print("\n=== Distribución del error por combinación SKU-mes ===")
valid = mensual_producto.dropna(subset=["error_pct"]).copy()
valid["abs_error_pct"] = valid["error_pct"].abs()
bandas = pd.cut(
    valid["abs_error_pct"],
    bins=[0, 0.20, 0.50, 1.0, np.inf],
    labels=["dentro de ±20%", "±20% a ±50%", "±50% a ±100%", "peor de ±100%"],
)
resumen_bandas = bandas.value_counts().sort_index()
for etiqueta, cuenta in resumen_bandas.items():

```

```

        print(f" {etiqueta}: {cuenta} de {len(valid)} combinaciones SKU-mes ({cuenta / len(valid):.1%})")

mensual_producto.to_csv(os.path.join(vdir, "backtest_mensual_producto.csv"), index=False)
mensual_total.to_csv(os.path.join(vdir, "backtest_mensual_total.csv"), index=False)
print(f"\nResultados del backtest guardados en: {vdir}")

pct_dentro_20 = float((valid["abs_error_pct"] <= 0.20).mean())
versioning.actualizar_resumen(version, {
    "dias_backtest": DIAS_BACKTEST,
    "wmape_backtest": round(w, 4),
    "sesgo_backtest": round(sesgo, 4),
    "pct_total_real_backtest": round(total_pred / total_real, 4),
    "pct_sku_mes_dentro_20pct": round(pct_dentro_20, 4),
})
print(f"\nResumen de versiones actualizado en: {versioning.SUMMARY_PATH}")

return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

sponser_etl/modeling/versioning.py

Gestión de versiones del modelo (versions/vN, LATEST.txt)

```

"""Utilidades para manejar versiones del modelo entrenado.

Cada corrida de train.py crea una carpeta nueva versions/vN/ (nunca sobrescribe
una anterior), con su modelo, encoders, configuración y métricas. versions/LATEST.txt
apunta a la más reciente, para que forecast.py y backtest.py la usen por defecto.
"""

from __future__ import annotations

import os
import re

import pandas as pd

VERSIONS_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "versions")
LATEST_PATH = os.path.join(VERSIONS_DIR, "LATEST.txt")
SUMMARY_PATH = os.path.join(VERSIONS_DIR, "summary.csv")

def listar_versiones() -> list[str]:
    if not os.path.isdir(VERSIONS_DIR):
        return []
    vs = [
        d for d in os.listdir(VERSIONS_DIR)
        if re.fullmatch(r"v\d+", d) and os.path.isdir(os.path.join(VERSIONS_DIR, d))
    ]
    return sorted(vs, key=lambda v: int(v[1:]))

def siguiente_version() -> str:
    vs = listar_versiones()
    if not vs:
        return "v1"
    return f"v{int(vs[-1][1:]) + 1}"

```

```

def version_dir(version: str) -> str:
    d = os.path.join(VERSIONS_DIR, version)
    os.makedirs(d, exist_ok=True)
    return d

def marcar_latest(version: str) -> None:
    os.makedirs(VERSIONS_DIR, exist_ok=True)
    with open(LATEST_PATH, "w", encoding="utf-8") as f:
        f.write(version)

def resolver_version(version: str | None = None) -> str:
    """Si version es None, devuelve la última entrenada (versions/LATEST.txt)."""
    if version:
        return version
    if not os.path.exists(LATEST_PATH):
        raise FileNotFoundError("No hay ninguna versión entrenada todavía. Corre train.py primero.")
    with open(LATEST_PATH, "r", encoding="utf-8") as f:
        return f.read().strip()

def actualizar_resumen(version: str, columnas: dict) -> None:
    """Agrega o actualiza (upsert) la fila de `version` en versions/summary.csv."""
    os.makedirs(VERSIONS_DIR, exist_ok=True)
    if os.path.exists(SUMMARY_PATH):
        df = pd.read_csv(SUMMARY_PATH)
    else:
        df = pd.DataFrame(columns=["version"])

    if "version" not in df.columns:
        df["version"] = []

    fila = {"version": version, **columnas}
    if version in df["version"].astype(str).values:
        idx = df.index[df["version"].astype(str) == version][0]
        for k, v in fila.items():
            df.loc[idx, k] = v
    else:
        df = pd.concat([df, pd.DataFrame([fila])], ignore_index=True)

    df.to_csv(SUMMARY_PATH, index=False)

```

sponser_etl/replenishment/fifo.py

Simulación FIFO de consumo de inventario y ofertas de proveedor

```

"""Simulación FIFO de consumo de inventario y asignación de ofertas de proveedor,
priorizando siempre lo que vence más pronto, para minimizar pérdida por vencimiento.
"""

from __future__ import annotations

from datetime import date, timedelta

import pandas as pd

def ajustar_a_moq_y_multiple(qty: float, moq: float, multiple: float) -> int:
    if qty <= 0:
        return 0
    qty = max(int(qty), int(moq or 0))

```

```

m = int(multiple or 0)
if m > 0 and qty % m != 0:
    qty += (m - qty % m)
return int(qty)

def rango_fechas(inicio: date, fin: date):
    dias = (fin - inicio).days
    for d in range(dias + 1):
        yield inicio + timedelta(days=d)

def construir_horizonte(sku_inv: pd.DataFrame, sku_ofertas: pd.DataFrame, lt: int, cov: int, saf: int,
hoy: date):
    base_fin = hoy + timedelta(days=max(0, int(lt) + int(cov) + int(saf)))
    max_exp_inv = max(sku_inv["expiry_date"]) if not sku_inv.empty else base_fin
    max_exp_of = max(sku_ofertas["expiry_date"]) if not sku_ofertas.empty else base_fin
    fin = max(base_fin, max_exp_inv, max_exp_of)
    fin = min(fin, hoy + timedelta(days=540)) # tope 18 meses
    return hoy, fin

def consumir_fifo_solo_inventario(demanda_diaria: pd.Series, sku_inv: pd.DataFrame, start_date: date,
end_date: date):
    """demanda_diaria: serie indexada por fecha con la curva día a día del
    pronóstico (no un promedio aplanado, para no perder picos por evento que
    puedan definir si un lote vence antes de venderse). Ver run.py."""
    fechas = list(rango_fechas(start_date, end_date))
    residual = pd.Series([float(demanda_diaria.get(f, 0.0)) for f in fechas], index=fechas, dtype=float)
    servido = pd.Series([0] * len(fechas), index=fechas, dtype=float)

    if sku_inv.empty or residual.sum() <= 0:
        return residual, 0, servido

    lots = sku_inv.sort_values("expiry_date").copy()
    lots_state = [{"expiry": r.expiry_date, "qty": float(r.qty)} for _, r in lots.iterrows()]

    for i, fecha in enumerate(fechas):
        demanda = residual.iloc[i]
        if demanda <= 0:
            continue
        for lot in lots_state:
            if demanda <= 0:
                break
            if lot["qty"] <= 0:
                continue
            if lot["expiry"] < fecha:
                continue
            usar = min(lot["qty"], demanda)
            lot["qty"] -= usar
            demanda -= usar
            servido.iloc[i] += usar
            residual.iloc[i] = demanda

    expirado = 0
    fin_h = end_date
    for lot in lots_state:
        if lot["qty"] > 0 and lot["expiry"] <= fin_h:
            expirado += lot["qty"]
            lot["qty"] = 0

    return residual, float(expirado), servido

def asignar_ofertas_a_residual(residual_series: pd.Series, opciones: pd.DataFrame):

```

```

"""Cubre la demanda residual (no cubierta por inventario actual) con ofertas
de proveedor ya negociadas, priorizando la que vence más pronto (igual criterio
que consumir_fifo_solo_inventario), y solo desde su fecha de llegada.

Devuelve: (asignado, residual_restante, riesgo)
- asignado: dict option_id -> unidades consumidas por la demanda.
- residual_restante: pd.Series, demanda que sigue sin cubrir después de aplicar ofertas.
- riesgo: dict option_id -> {"at_risk": unidades de la oferta que quedaron sin usar
    al llegar su vencimiento (desperdicio potencial)}.
"""
residual = residual_series.copy()
asignado = {opt: 0.0 for opt in opciones["option_id"]}
riesgo = {opt: {"at_risk": 0.0} for opt in opciones["option_id"]}

if opciones.empty:
    return asignado, residual, riesgo

opciones_sorted = opciones.sort_values("expiry_date").copy()

for _, row in opciones_sorted.iterrows():
    opt_id = row["option_id"]
    arr = row["arrival_date"]
    exp = row["expiry_date"]
    disponible = float(row["available_qty"])

    for fecha in residual.index:
        if disponible <= 0:
            break
        if fecha < arr or fecha > exp:
            continue
        demanda = residual.loc[fecha]
        if demanda <= 0:
            continue
        usar = min(disponible, demanda)
        residual.loc[fecha] -= usar
        disponible -= usar
        asignado[opt_id] += usar

    if disponible > 0:
        riesgo[opt_id]["at_risk"] = disponible

return asignado, residual, riesgo

```

sponser_etl/replenishment/run.py

Orquestador de la recomendación de compra

```

"""Orquestador: demanda pronosticada + inventario real + parámetros de compra
-> recomendación de compra por producto (FIFO anti-vencimiento).

Uso: python run.py (desde sponser_etl/replenishment/)
"""

from __future__ import annotations

import os
import sys

import pandas as pd

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__)))) # sponser_etl/ (para
load.py)

```

```

import load
import fifo

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__))) # sponser_etl/
PROCESSED_DIR = os.path.join(BASE_DIR, "data", "processed")

ADVERTENCIA = """
=====
ADVERTENCIA - revisar antes de usar para decisiones de compra:
1. El pronóstico (modelo hurdle, calibrado por tercil de volumen) tiene un margen de error real a nivel de SKU individual, mayor cuanto más chico es el volumen del producto – ver modeling/versions/summary.csv y versions/LATEST/backtest_resultados.csv (LATEST.txt indica cuál es la versión vigente) antes de confiar en un número exacto por SKU chico.
2. sku_config.csv arrancó con valores PLACEHOLDER de lead time / MOQ / múltiplo (no son datos reales del proveedor Sponser) – corrígelos ahí antes de confiar en el resultado.
3. ofertas_proveedor.csv está vacío: se asume que no hay ningún pedido ya en camino: toda la demanda no cubierta por el inventario actual aparece como "a pedir".
=====
"""

def cargar_as_of_date():
    """Última fecha con venta real registrada -- misma ancla temporal que usa sponser_api (data_access.get_as_of_date()) para todo el resto del tablero."""
    ventas = pd.read_csv(
        os.path.join(PROCESSED_DIR, "ventas_diarias_producto.csv"), parse_dates=["fecha"]
    )
    return ventas["fecha"].max().date()

def cargar_inputs():
    demanda = pd.read_csv(
        os.path.join(PROCESSED_DIR, "demanda_promedio_sku.csv"), dtype={"codigo_producto": str}
    )

    # Curva día a día del pronóstico; demanda_promedio_sku.csv (arriba) es su # promedio, usado como respaldo (ver construir_demanda_diaria).
    forecast = pd.read_csv(
        os.path.join(PROCESSED_DIR, "forecast_demanda_producto.csv"),
        dtype={"codigo_producto": str}, parse_dates=["fecha"],
    )
    forecast["fecha"] = forecast["fecha"].dt.date
    forecast_por_sku = {
        codigo: grupo.set_index("fecha")["unidades_pronosticadas"]
        for codigo, grupo in forecast.groupby("codigo_producto")
    }

    inv = pd.read_csv(
        os.path.join(PROCESSED_DIR, "inventario_lotes.csv"), dtype={"codigo_producto": str}
    )
    inv = inv.rename(columns={"codigo_producto": "sku", "lote_id": "lot_id", "cantidad": "qty"})
    inv["expiry_date"] = pd.to_datetime(inv["fecha_vencimiento"], errors="coerce")
    fecha_lejana = pd.Timestamp.today() + pd.Timedelta(days=3650)
    inv["expiry_date"] = inv["expiry_date"].fillna(fecha_lejana).dt.date

    cfg = pd.read_csv(os.path.join(BASE_DIR, "sku_config.csv"), dtype={"codigo_producto": str})

    ofertas = pd.read_csv(os.path.join(BASE_DIR, "ofertas_proveedor.csv"), dtype={"sku": str})
    if not ofertas.empty:

```

```

    ofertas["arrival_date"] = pd.to_datetime(ofertas["arrival_date"]).dt.date
    ofertas["expiry_date"] = pd.to_datetime(ofertas["expiry_date"]).dt.date

    return demanda, forecast_por_sku, inv, cfg, ofertas

def cargar_descripciones() -> pd.Series:
    """codigo_producto -> descripción más frecuente en ventas_detalle.csv (cubre
    los 175 productos con demanda, a diferencia de inventario_lotes.csv que solo
    tiene descripción para los que además tienen stock registrado)."""
    ventas = pd.read_csv(
        os.path.join(PROCESSED_DIR, "ventas_detalle.csv"),
        dtype={"codigo_producto": str},
        usecols=["codigo_producto", "item_descripcion"],
    )
    ventas["descripcion"] = ventas["item_descripcion"].astype(str).str.replace(
        r"^\s*\d+\s*", "", regex=True
    ).str.strip()
    return ventas.groupby("codigo_producto")["descripcion"].agg(lambda s: s.mode().iat[0])

def construir_demanda_diaria(daily_avg: float, curva: pd.Series | None, start, end) -> pd.Series:
    """Serie día a día del horizonte de compra: usa la curva pronosticada
    donde exista, y daily_avg de respaldo para los días fuera de ese rango
    (el horizonte puede extenderse más que el pronóstico -- ver
    fifo.construir_horizonte)."""
    fechas = list(fifo.rango_fechas(start, end))
    if curva is None or curva.empty:
        return pd.Series([daily_avg] * len(fechas), index=fechas, dtype=float)
    valores = [float(curva[f]) if f in curva.index else daily_avg for f in fechas]
    return pd.Series(valores, index=fechas, dtype=float)

def calcular_recomendacion(
    demanda: pd.DataFrame, forecast_por_sku: dict, inv: pd.DataFrame, cfg: pd.DataFrame, ofertas:
    pd.DataFrame, hoy,
) -> pd.DataFrame:
    cfg_idx = cfg.set_index("codigo_producto")
    filas = []

    for _, row in demanda.iterrows():
        codigo = row["codigo_producto"]
        daily_avg = float(row["daily_demand"])

        if codigo in cfg_idx.index:
            params = cfg_idx.loc[codigo]
            lt, cov, saf = params["lead_time_days"], params["coverage_days"], params["safety_days"]
            moq, multiple = params["moq"], params["multiple"]
        else:
            lt, cov, saf, moq, multiple = 21, 30, 7, 1, 1

        sku_inv = inv[inv["sku"] == codigo][["lot_id", "qty", "expiry_date"]]
        sku_ofertas = ofertas[ofertas["sku"] == codigo] if not ofertas.empty else ofertas

        start, end = fifo.construir_horizonte(sku_inv, sku_ofertas, lt, cov, saf, hoy)
        demanda_diaria = construir_demanda_diaria(daily_avg, forecast_por_sku.get(codigo), start, end)
        residual, expirado, _servido = fifo.consumir_fifo_solo_inventario(demanda_diaria, sku_inv, start,
end)

        asignado, residual_final, riesgo = fifo.asignar_ofertas_a_residual(residual, sku_ofertas)

        demanda_no_cubierta = float(residual_final.clip(lower=0).sum())
        unidades_a_pedir = fifo.ajustar_a_moq_y_multiple(demanda_no_cubierta, moq, multiple)
        riesgo_ofertas = sum(r["at_risk"] for r in riesgo.values())

        filas.append({

```

```

        "codigo_producto": codigo,
        "daily_demand": daily_avg,
        "dias_horizonte": (end - start).days + 1,
        "unidades_en_inventario": float(sku_inv["qty"].sum()) if not sku_inv.empty else 0.0,
        "unidades_en_riesgo_vencimiento": float(expirado) + float(riesgo_ofertas),
        "unidades_cubiertas_por_ofertas": float(sum(asignado.values())),
        "unidades_a_pedir": unidades_a_pedir,
    })

return pd.DataFrame(filas).sort_values("unidades_a_pedir", ascending=False).reset_index(drop=True)

def main() -> int:
    print(ADVERTENCIA)

    demanda, forecast_por_sku, inv, cfg, ofertas = cargar_inputs()
    hoy = cargar_as_of_date()
    print(f"Calculando recomendación de compra para {len(demanda)} productos (referencia: hoy = {hoy},
última venta real registrada)...")

    recomendacion = calcular_recomendacion(demanda, forecast_por_sku, inv, cfg, ofertas, hoy)

    descripciones = cargar_descripciones()
    recomendacion.insert(1, "descripcion", recomendacion["codigo_producto"].map(descripciones).fillna(""))

    load.guardar_tablas({"recomendacion_compra": recomendacion}, PROCESSED_DIR)

    print(f"\nrecomendacion_compra.csv: {len(recomendacion):,} filas -> {PROCESSED_DIR}")
    print(f"Productos con unidades_a_pedir > 0: {(recomendacion['unidades_a_pedir'] > 0).sum()}")
    print(f"Total unidades a pedir (todos los productos): {recomendacion['unidades_a_pedir'].sum():,}.0f")

    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```